Pseudocode and Analyzing Runtime

Prof. Timothy Sun

A personal take on pseudocode

For homework and exams, all algorithms must be described in ***pseudocode***:

- structure of some programming languages (blocks, loops, arrays, etc.)
- natural languages (e.g. “and” instead of “&&”)
- mathematical notation (sets, arithmetic operators, etc.)

Don’t submit actual code (even Python)!

A personal take on pseudocode

```java
public void sort(int array[]){
    int n = array.length;
    for (int i = 1; i < n; i++){
        int key = array[i];
        int j = i - 1;
        while (j >= 0 && array[j] > key){
            array[j + 1] = array[j];
            j--;
        }
        array[j + 1] = key;
    }
}
```

$\textsc{InsertionSort}(A[1,\ldots,n])$:

$\quad$ **for** $i = 2,\ldots,n$

$\quad\quad$ $key := A[i]$

$\quad\quad$ $j := i - 1$

$\quad\quad$ **while** $j \geq 1$ and $A[j] > key$

$\quad\quad\quad$ $A[j+1] := A[j]$

$\quad\quad\quad$ $j := j - 1$

$\quad\quad$ $A[j+1] := key$

Assignment

```
int j = i - 1;
```

Assignment is written as…

$$j := i - 1$$

to distinguish it from “equals”.

No need to declare types

Arrays

```
int array[]
int n = array.length;
```

Declare arrays in the following way:

$$A[1, \ldots, n]$$

A[1] and ***A***[n] are defined (sorry…)

(It saves a line!)

Loops and blocks

```
for (int i = 1; i < n; i++){

}
```

For loops will be “for-each” loops:

$$\textbf{for } i = 2, \ldots, n$$

No need for parentheses, etc.

Blocks denoted by indentation alone.

Boolean and arithmetic operators

```
while (j >= 0 && array[j] > key){
```

Boolean operators replaced with words:

$$\textbf{while } j \geq 1 \text{ and } A[j] > key$$

Use of math symbols when appropriate.

Function declarations

```
public void sort(int array[]){
}
```

Once again, no types, just the name and parameters

$$\textsc{InsertionSort}(A[1,\ldots,n]):$$

A personal take on pseudocode

```java
public void sort(int array[]){
    int n = array.length;
    for (int i = 1; i < n; i++){
        int key = array[i];
        int j = i - 1;
        while (j >= 0 && array[j] > key){
            array[j + 1] = array[j];
            j--;
        }
        array[j + 1] = key;
    }
}
```

$\textsc{InsertionSort}(A[1,\ldots,n])$:

$\quad$ **for** $i = 2,\ldots,n$

$\quad\quad$ $key := A[i]$

$\quad\quad$ $j := i - 1$

$\quad\quad$ **while** $j \geq 1$ and $A[j] > key$

$\quad\quad\quad$ $A[j+1] := A[j]$

$\quad\quad\quad$ $j := j - 1$

$\quad\quad$ $A[j+1] := key$

Runtime of iterative algorithms

Real algorithms have many different kinds of operations, so let's look at some ***toy examples*** that use a "count" variable that plays the role of time:

```
InsertionSort(A[1,...,n]):
    for i = 2,...,n
        key := A[i]
        j := i − 1
        while j ≥ 1 and A[j] > key
            A[j + 1] := A[j]
            j := j − 1
        A[j + 1] := key
```

```
Function(n):
    count := 0
    for i = 1,...,n
        for j = i,...,n
            count := count + 1
    return count
```

No loops

Give a big-O estimate for the return value of “count”:

$$
\begin{aligned}
&\textsc{Function}(n):\\
&\qquad count := 0\\
&\qquad count := count + 1\\
&\qquad \textbf{return } count
\end{aligned}
$$

No loops

Give a big-O estimate for the return value of “count”:

$$
\begin{aligned}
&\textsc{Function}(n):\\
&\qquad count := 0\\
&\qquad count := count + 1\\
&\qquad \textbf{return } count
\end{aligned}
$$

Answer: O(1)

A single loop

Give a big-O estimate for the return value of “count”:

$$
\begin{aligned}
&\textsc{Function}(n):\\
&\qquad count := 0\\
&\qquad \textbf{for } i = 1, \ldots, n\\
&\qquad\qquad count := count + 1\\
&\qquad \textbf{return } count
\end{aligned}
$$

A single loop

Give a big-O estimate for the return value of “count”:

$$
\begin{aligned}
&\textsc{Function}(n):\\
&\qquad count := 0\\
&\qquad \textbf{for } i = 1, \ldots, n\\
&\qquad\qquad count := count + 1\\
&\qquad \textbf{return } count
\end{aligned}
$$

Answer: O(***n***), because the loop does ***n*** iterations

Different number of iterations

Give a big-O estimate for the return value of “count”:

$$
\begin{aligned}
&\textsc{Function}(n):\\
&\qquad count := 0\\
&\qquad \textbf{for } i = 1, \ldots, 2n\\
&\qquad\qquad count := count + 1\\
&\qquad \textbf{return } count
\end{aligned}
$$

Different number of iterations

Give a big-O estimate for the return value of “count”:

$$
\begin{aligned}
&\textsc{Function}(n):\\
&\qquad count := 0\\
&\qquad \textbf{for } i = 1, \ldots, 2n\\
&\qquad\qquad count := count + 1\\
&\qquad \textbf{return } count
\end{aligned}
$$

Answer: O(***n***), again

Different number of iterations

Give a big-O estimate for the return value of “count”:

$$
\begin{aligned}
&\textsc{Function}(n):\\
&\qquad count := 0\\
&\qquad \textbf{for } i = 1, \ldots, n^3\\
&\qquad\qquad count := count + 1\\
&\qquad \textbf{return } count
\end{aligned}
$$

Different number of iterations

Give a big-O estimate for the return value of “count”:

$$
\begin{aligned}
&\textsc{Function}(n):\\
&\qquad count := 0\\
&\qquad \textbf{for } i = 1, \ldots, n^3\\
&\qquad\qquad count := count + 1\\
&\qquad \textbf{return } count
\end{aligned}
$$

Answer: $O(\boldsymbol{n}^3)$

Exponentially growing variables

Give a big-O estimate for the return value of “count”:

```
FUNCTION(n):
    count := 0
    i := 1
    while i < n:
        count := count + 1
        i := 2i
    return count
```

Exponentially growing variables

Give a big-O estimate for the return value of “count”:

```
FUNCTION(n):
    count := 0
    i := 1
    while i < n:
        count := count + 1
        i := 2i
    return count
```

Answer: O(log ***n***)

Consecutive loops

Give a big-O estimate for the return value of “count”:

$$
\begin{aligned}
&\textsc{Function}(n):\\
&\qquad count := 0\\
&\qquad \textbf{for } i = 1, \ldots, n\\
&\qquad\qquad count := count + 1\\
&\qquad \textbf{for } i = 1, \ldots, n\\
&\qquad\qquad count := count + 1\\
&\qquad \textbf{return } count
\end{aligned}
$$

Consecutive loops

Give a big-O estimate for the return value of “count”:

$$
\begin{aligned}
&\textsc{Function}(n):\\
&\qquad count := 0\\
&\qquad \textbf{for } i = 1, \ldots, n\\
&\qquad\qquad count := count + 1\\
&\qquad \textbf{for } i = 1, \ldots, n\\
&\qquad\qquad count := count + 1\\
&\qquad \textbf{return } count
\end{aligned}
$$

Answer: O(***n***)+O(***n***) = O(***n***)

Nested loops

Give a big-O estimate for the return value of “count”:

$$
\begin{aligned}
&\textsc{Function}(n):\\
&\qquad count := 0\\
&\qquad \textbf{for } i = 1, \ldots, n\\
&\qquad\qquad \textbf{for } j = 1, \ldots, n\\
&\qquad\qquad\qquad count := count + 1\\
&\qquad \textbf{return } count
\end{aligned}
$$

Nested loops

Give a big-O estimate for the return value of “count”:

$$
\begin{array}{l}
\textsc{Function}(n): \\
\quad count := 0 \\
\quad \textbf{for } i = 1, \ldots, n \\
\qquad \textbf{for } j = 1, \ldots, n \\
\qquad\quad count := count + 1 \\
\quad \textbf{return } count
\end{array}
$$

Answer: O(***n***)O(***n***) = O(***n***^{2}), since each loop runs for ***n*** iterations

Nested loops

Give a big-O estimate for the return value of “count”:

$$
\begin{aligned}
&\textsc{Function}(n):\\
&\qquad count := 0\\
&\qquad \textbf{for } i = 1, \ldots, n\\
&\qquad\qquad \textbf{for } j = 1, \ldots, n\\
&\qquad\qquad\qquad \textbf{for } k = 1, \ldots, n\\
&\qquad\qquad\qquad\qquad count := count + 1\\
&\qquad \textbf{return } count
\end{aligned}
$$

Nested loops

Give a big-O estimate for the return value of “count”:

$$
\begin{aligned}
&\textsc{Function}(n):\\
&\qquad count := 0\\
&\qquad \textbf{for } i = 1, \ldots, n\\
&\qquad\qquad \textbf{for } j = 1, \ldots, n\\
&\qquad\qquad\qquad \textbf{for } k = 1, \ldots, n\\
&\qquad\qquad\qquad\qquad count := count + 1\\
&\qquad \textbf{return } count
\end{aligned}
$$

Answer: $O(\boldsymbol{n}^3)$

Two kinds of loops

consecutive loops = add

$\textsc{Function}(n)$:

$count := 0$

for $i = 1, \ldots, n$

$count := count + 1$

for $i = 1, \ldots, n$

$count := count + 1$

return $count$

Answer: O(***n***)

nested loops = multiply

$\textsc{Function}(n)$:

$count := 0$

for $i = 1, \ldots, n$

for $j = 1, \ldots, n$

$count := count + 1$

return $count$

Answer: O($\boldsymbol{n}^2$)

Nested, consecutive, nested…

```
FUNCTION(n):
    count := 0
    for i = 1, ..., n
        for j = 1, ..., n
            for k = 1, ..., n
                count := count + 1
        for ℓ = 1, ..., n
            count := count + 1
    return count
```

Nested, consecutive, nested…

```
FUNCTION(n):
    count := 0
    for i = 1, ..., n
        for j = 1, ..., n
            O(n)
        for ℓ = 1, ..., n
            count := count + 1
    return count
```

Nested, consecutive, nested…

```
FUNCTION(n):
    count := 0
    for i = 1, ..., n
        O(n)O(n)
        for ℓ = 1, ..., n
            count := count + 1
    return count
```

Nested, consecutive, nested…

$\textsc{Function}(n)$:

$\quad count := 0$

$\quad \textbf{for } i = 1, \ldots, n$

$\qquad O(n)O(n)$

$\qquad O(n)$

$\quad \textbf{return } count$

Nested, consecutive, nested…

$\textsc{Function}(n)$:

$\quad count := 0$

$\quad \textbf{for } i = 1, \ldots, n$

$\qquad O(n)O(n) + O(n)$

$\quad \textbf{return } count$

Nested, consecutive, nested…

$\textsc{Function}(n)$:

$$O(n)[O(n)O(n) + O(n)]$$

Nested, consecutive, nested…

```
FUNCTION(n):
    count := 0
    for i = 1, ..., n
        for j = 1, ..., n
            for k = 1, ..., n
                count := count + 1
        for ℓ = 1, ..., n
            count := count + 1
    return count
```

Answer: $O(\boldsymbol{n})[O(\boldsymbol{n})O(\boldsymbol{n})+O(\boldsymbol{n})] = O(\boldsymbol{n}^3)$

Varying number of iterations?

Give a big-O estimate for the return value of “count”:

$$
\begin{array}{l}
\textsc{Function}(n): \\
\quad count := 0 \\
\quad \textbf{for } i = 1, \ldots, n \\
\qquad \textbf{for } j = i, \ldots, n \\
\qquad\quad count := count + 1 \\
\quad \textbf{return } count
\end{array}
$$

Varying number of iterations?

Give a big-O estimate for the return value of “count”:

$$
\begin{array}{l}
\textsc{Function}(n): \\
\quad count := 0 \\
\quad \textbf{for } i = 1, \ldots, n \\
\qquad \textbf{for } j = i, \ldots, n \\
\qquad\quad count := count + 1 \\
\quad \textbf{return } count
\end{array}
$$

Answer: still O($\boldsymbol{n}^2$)

Estimating difficult sums

$n + \ldots + 1 = n(n+1)/2 = \Theta(n^2)$, but what if we didn't know the formula?

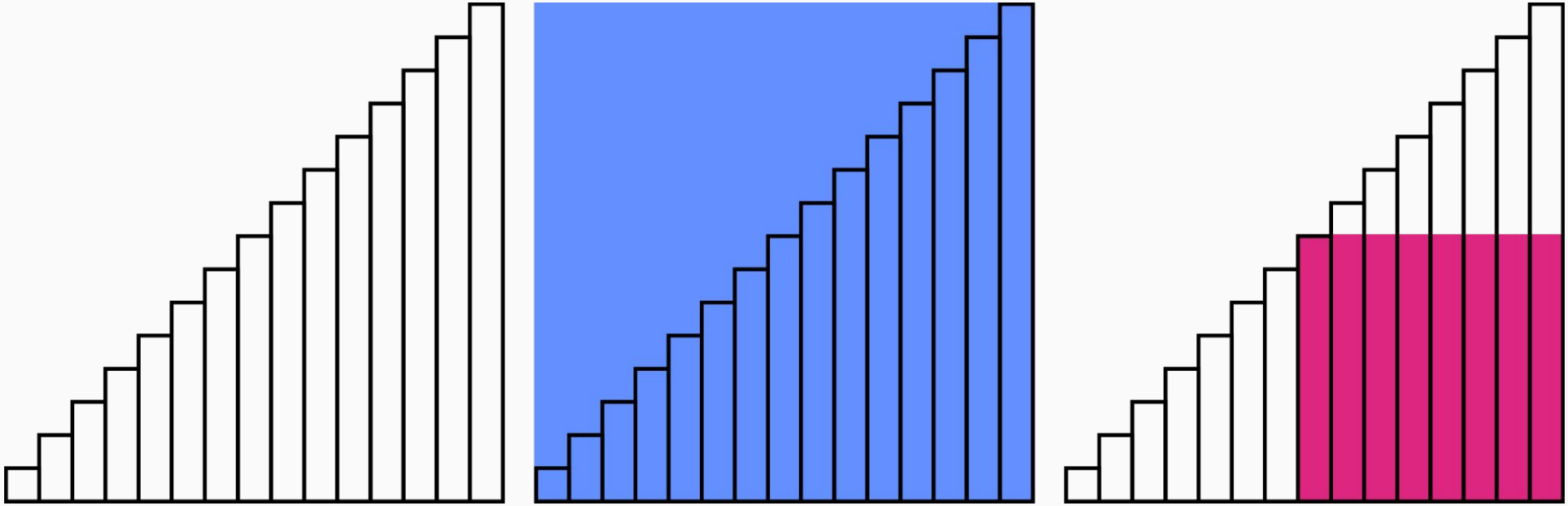
$$
\begin{aligned}
& 1 + 2 + 3 + 4 + \ldots + n \\
\leq\ & n + n + n + n + \ldots + n \\
=\ & n^2
\end{aligned}
$$

Estimating difficult sums

$n + \ldots + 1 = n(n+1)/2 = \Theta(n^2)$, but what if we didn't know the formula?

$$
\begin{aligned}
& 1 + 2 + 3 + 4 + \ldots n/2 + \ldots + n \\
\geq \; & n/2 + \ldots + n \\
\geq \; & n/2 + \ldots + n/2 \\
\approx \; & (n/2)^2
\end{aligned}
$$

A useful estimation trick



Function calls

Give a big-O estimate for the runtime of “Function”:

```
SUBROUTINE(n):
    count := 0
    for i = 1, ..., n
        count := count + 1
    return count
FUNCTION(n):
    count := 0
    for i = 1, ..., n
        count := count + SUBROUTINE(n)
    return count
```

Function calls

Give a big-O estimate for the runtime of “Function”:

```
SUBROUTINE(n):
    count := 0
    for i = 1, ..., n
        count := count + 1
    return count
FUNCTION(n):
    count := 0
    for i = 1, ..., n
        count := count + SUBROUTINE(n)
    return count
```

Answer: $O(\boldsymbol{n}^2)$,
since “Subroutine” is called $O(\boldsymbol{n})$ times

Analyzing iterative algorithms

Give a big-O estimate for the return value of “count”:

```
FUNCTION(m, n):
    count := 0
    for i = 1, ..., m
        for j = 1, ..., n
            count := count + 1
    return count
```

Analyzing iterative algorithms

Give a big-O estimate for the return value of “count”:

$$
\begin{array}{l}
\textsc{Function}(m, n): \\
\quad count := 0 \\
\quad \textbf{for } i = 1, \ldots, m \\
\qquad \textbf{for } j = 1, \ldots, n \\
\qquad\quad count := count + 1 \\
\quad \textbf{return } count
\end{array}
$$

Answer: two variables now, O(***mn***)

Putting it all together

Give a big-O estimate for the runtime of “Function”:

```
FUNCTION(m, n):
    count := 0
    for i = 1, ..., m
        for j = 1, ..., i
            for k = 1, ..., n
                count := count + 1
    for ℓ = 1, ..., 3n
        count := count + SUBROUTINE(m)
    return count
```

```
SUBROUTINE(n):
    count := 0
    i := 1
    while i ≤ n
        count := count + 1
        i := i + 1
    return count
```

Putting it all together

Give a big-O estimate for the runtime of "Function":

$O(n)$

```
FUNCTION(m, n):
    count := 0
    for i = 1, ..., m
        for j = 1, ..., i
            for k = 1, ..., n
                count := count + 1
    for ℓ = 1, ..., 3n
        count := count + SUBROUTINE(m)
    return count
```

```
SUBROUTINE(n):
    count := 0
    i := 1
    while i ≤ n
        count := count + 1
        i := i + 1
    return count
```

Putting it all together

Give a big-O estimate for the runtime of "Function":

$O(n)$

$\textsc{Function}(m, n)$:

$count := 0$

for $i = 1, \ldots, m$

for $j = 1, \ldots, i$

$O(n)$ **for** $k = 1, \ldots, n$

$count := count + 1$

for $\ell = 1, \ldots, 3n$

$count := count + \textsc{Subroutine}(m)$

return $count$

$O(m)$

$\textsc{Subroutine}(n)$:

$count := 0$

$i := 1$

while $i \leq n$

$count := count + 1$

$i := i + 1$

return $count$

Putting it all together

Give a big-O estimate for the runtime of "Function":

$O(n)$

```
FUNCTION(m, n):
    count := 0
    for i = 1, ..., m
        for j = 1, ..., i
            for k = 1, ..., n
                count := count + 1
    for ℓ = 1, ..., 3n
        count := count + SUBROUTINE(m)
    return count
```

$O(n)$

$O(m)$

```
SUBROUTINE(n):
    count := 0
    i := 1
    while i ≤ n
        count := count + 1
        i := i + 1
    return count
```

Answer: $O(m)O(m)O(n)+O(n)O(m) = O(m^2n)$

Different kinds of operations

Treat each primitive operation (assignment, comparison, math operations, etc.) as O(1) time.

$\textsc{InsertionSort}(A[1,\ldots,n])$:

for $i = 2,\ldots,n$

$key := A[i]$

$j := i - 1$

while $j \geq 1$ and $A[j] > key$

$A[j+1] := A[j]$

$j := j - 1$

$A[j+1] := key$

O(1)

Different kinds of operations

Treat each primitive operation (assignment, comparison, math operations, etc.) as O(1) time.

Can use previous rules to combine into one simple expression.

$\textsc{InsertionSort}(A[1,\ldots,n])$:

for $i = 2,\ldots,n$

$key := A[i]$

$j := i - 1$

while $j \geq 1$ and $A[j] > key$

$A[j+1] := A[j]$

$j := j - 1$

$A[j+1] := key$

O(1)

Different kinds of operations

while loop: worst case O(***n***) iterations
for loop: ***n*** - 1 = O(***n***) iterations

$$
\begin{array}{l}
\textsc{InsertionSort}(A[1,\ldots,n]): \\
\quad \textbf{for } i = 2,\ldots,n \\
\qquad key := A[i] \\
\qquad j := i - 1 \\
\qquad \textbf{while } j \geq 1 \text{ and } A[j] > key \\
\qquad\quad A[j+1] := A[j] \\
\qquad\quad j := j - 1 \\
\qquad A[j+1] := key
\end{array}
$$

Different kinds of operations

while loop: worst case O(***n***) iterations
for loop: ***n*** - 1 = O(***n***) iterations

Answer:

O(***n***)[O(1)+O(***n***)O(1)+O(1)]

$$
\begin{array}{l}
\textsc{InsertionSort}(A[1,\ldots,n]): \\
\quad \textbf{for } i = 2,\ldots,n \\
\qquad key := A[i] \\
\qquad j := i-1 \\
\qquad \textbf{while } j \geq 1 \text{ and } A[j] > key \\
\qquad\quad A[j+1] := A[j] \\
\qquad\quad j := j-1 \\
\qquad A[j+1] := key
\end{array}
$$

Different kinds of operations

while loop: worst case O(***n***) iterations
for loop: ***n*** - 1 = O(***n***) iterations

Answer:

O(***n***)[O(1)+O(***n***)O(1)+O(1)]

= O(n^2) worst case runtime

$$
\begin{aligned}
&\textsc{InsertionSort}(A[1,\ldots,n]):\\
&\quad \textbf{for } i = 2,\ldots,n\\
&\qquad key := A[i]\\
&\qquad j := i-1\\
&\qquad \textbf{while } j \geq 1 \text{ and } A[j] > key\\
&\qquad\quad A[j+1] := A[j]\\
&\qquad\quad j := j-1\\
&\qquad A[j+1] := key
\end{aligned}
$$

Different kinds of operations

Don't forget to include the cost of any function calls, if any.

```
SUBROUTINE(n):
    count := 0
    for i = 1, ..., n
        count := count + 1
    return count
FUNCTION(n):
    count := 0
    for i = 1, ..., n
        count := count + SUBROUTINE(n)
    return count
```